COMPARISON

# Convert HTML to PDF in Node.js: which route fits

Updated 12 Jan 2027

KEY TAKEAWAYS

- To turn HTML and CSS into a PDF the way a browser renders them, drive headless Chromium with Playwright or Puppeteer.
- PDFKit and pdfmake build a PDF from code or a document definition. Neither README lists HTML as an input.
- html-pdf wraps PhantomJS. Its GitHub repository was archived on 8 July 2026 and npm marks the package deprecated.
- Gotenberg is the same Chromium route as a Docker service: you POST files and get a PDF back.
- A template needs no route of its own. Render it to an HTML string with any engine, then print that.

CONTENTS

1 HTML to PDF converters for Node.js side by side ..... 3
2 Playwright ..... 4
3 Puppeteer ..... 4
4 PDFKit and pdfmake: no HTML ..... 5
5 The html-pdf package ..... 6
6 Gotenberg ..... 6
7 HTML template to PDF in Node.js: render, then print ..... 7
8 Where papyrus fits ..... 8
9 What to remember ..... 8

To convert HTML to PDF in Node.js with the HTML and CSS rendered the way a browser renders them, drive headless Chromium with Playwright or Puppeteer. Libraries that draw PDFs themselves, such as PDFKit and pdfmake, don’t take HTML: you build the document in code. Older HTML wrappers are a third route, and the best known, html-pdf, is archived and deprecated. Gotenberg puts the Chromium route behind a Docker service.

So if you’re after a JavaScript library to convert HTML to PDF, the real choice is between one that drives a browser and one that draws pages itself. A rule of thumb: if the document already exists as HTML you can open, use Chromium. If you have data and want to control every line of the layout, a drawing library may be less work than HTML. If you don’t want a browser inside your app, run Gotenberg next to it.

This page compares the routes on what each project says about itself, as of 11 November 2026. papyrus and n-uf are not affiliated with Microsoft (Playwright), Google (Puppeteer, Chrome), the Gotenberg project, or the authors of PDFKit, pdfmake and html-pdf.

## 1. HTML to PDF converters for Node.js side by side

| Route | Takes HTML and CSS? | Engine | Runs where | Page breaks | Latest release (as of 11 Nov 2026) | Typical use |
|---|---|---|---|---|---|---|
| Playwright | Yes | Chromium (`page.pdf()` works only in Chromium) | A browser your Node process launches | Print CSS | 1.65.0, 5 Nov 2026 (npm) | Print any page you can open |
| Puppeteer | Yes | Chrome (it can also drive Firefox) | A browser your Node process launches | Print CSS | 25.17.0, 10 Nov 2026 (npm) | The same job, with its own API |
| Gotenberg | Yes | Chromium; LibreOffice for office files | A Docker container with an HTTP API | Print CSS | v9.4.0, 10 Nov 2026 (GitHub) | PDF as a service, for any language |
| PDFKit | No | Its own drawing code | Node and the browser | You call `addPage()` | 0.20.2, 30 Aug 2026 (npm) | Documents built in code |
| pdfmake | No | Its own, from a document definition | Node and the browser | Custom breaks; repeated table headers | 0.3.14, 8 Nov 2026 (npm) | Tables, headers and footers from data |
| html-pdf | Yes | PhantomJS | Node | Not covered in its README | 3.0.1, 7 May 2021; archived and deprecated | Existing code only |

“Print CSS” means you control breaks with CSS, as you would in a browser; css page break print shows the rules. The release dates tell you what’s current, not what’s good.

## 2. Playwright

Playwright's README calls it a framework for web automation and testing that drives Chromium, Firefox and WebKit with a single API. `page.pdf()` works only in Chromium, as Playwright's MCP PDF tool page says. It prints with the page's print CSS and returns the PDF as a buffer, and writes it to disk too if you pass a `path`. You install it with `npm i -D playwright` and `npx playwright install chromium`.

The script, the options, headers and footers, and a page-count check are in Playwright: generate PDF from HTML, so they aren't repeated here. For choosing, what matters is the shape of the route: one real browser engine, driven from your Node code, printing the page as its print CSS says.

**Where it fits:** a server or CI step that prints a page you can open by URL, or an HTML string, with the CSS the browser already runs. Playwright also has a Python version, per its docs, if part of your stack isn't Node.

**Where it doesn't:** it's a general browser automation tool, so a browser has to be installed wherever the job runs, and it prints only from Chromium.

---

## 3. Puppeteer

Puppeteer's home page describes a JavaScript library with a high-level API to control Chrome or Firefox over the DevTools Protocol or WebDriver BiDi, headless by default. `npm i puppeteer` downloads a compatible Chrome; `puppeteer-core` doesn't. The same page notes that modern package managers, npm among them, block dependency install scripts by default; then no browser is downloaded, and you run `npx puppeteer browsers install`. Its PDF guide says `Page.pdf()` waits for fonts to load by default.

```
import puppeteer from "puppeteer";

const browser = await puppeteer.launch();
const page = await browser.newPage();
await page.setContent("<h1>Invoice 1042</h1>");
await page.pdf({ path: "invoice.pdf", format: "A4" });
await browser.close();
```

We ran this with `puppeteer-core` 25.17.0 and the installed Chrome (`channel: "chrome"`) instead of the download, and got a one-page A4 PDF. Puppeteer does the Playwright job with a different API. **Where it fits** is the same as Playwright, and if you already use it for something else, that's reason enough to stay. **Where it doesn't:** the same limit as any route that needs a browser installed.

## 4. PDFKit and pdfmake: no HTML

PDFKit's README describes a JavaScript PDF generation library for Node and the browser. Its feature list covers vector graphics, text, fonts, images, tables, annotations and more. You make the document with calls, and the README's example uses `addPage()` for a second page.

```
import { PDFDocument } from "pdfkit";
import { createWriteStream } from "node:fs";

const doc = new PDFDocument({ size: "A4", margin: 50 });
doc.pipe(createWriteStream("invoice.pdf"));
doc.fontSize(20).text("Invoice 1042");
doc.end();
```

pdfmake builds the PDF from a document definition object. Its README lists table headers repeated after a page break, custom page breaks, and page headers and footers that can read the current page number and the page count. This is its Node quick start:

```
const pdfmake = require('pdfmake');
const Roboto = require('pdfmake/fonts/Roboto');

pdfmake.addFonts(Roboto);

const docDefinition = {
  content: ['Hello from pdfmake!']
};

pdfmake.createPdf(docDefinition).write('document.pdf').catch(console.error);
```

We ran both, with PDFKit 0.20.2 and pdfmake 0.3.14, and each gave a one-page PDF.

**Where they fit:** documents built from data in code, where you want exact control of the page and no browser. pdfmake's page numbers and repeated table headers come built in, where a browser route makes you assemble them from CSS (css print page numbers covers that side).

**Where they don't:** their READMEs list no HTML input, so if the layout already exists as HTML and CSS, you'd rebuild it as calls or a definition.

## 5. The html-pdf package

html-pdf is the old answer to this question. Its README describes an HTML to PDF converter that uses PhantomJS. The repository's GitHub page says: "This repository was archived by the owner on Jul 8, 2026. It is now read-only." The README adds that the repo isn't maintained anymore because PhantomJS was deprecated, and tells you to migrate to headless Chrome or Puppeteer. On npm, the latest version is 3.0.1, published on 7 May 2021, and it's marked deprecated with a message to move to a newer library such as Puppeteer.

**Where it fits:** code that already uses it, until you move. **Where it doesn't:** anything new. The html-pdf node alternative is the Chromium route above.

## 6. Gotenberg

Gotenberg's README calls it a Docker-based API for PDF that converts web pages and office documents. Its docs split the work into Chromium routes, for web pages, HTML and Markdown, and LibreOffice routes for Word, Excel, PowerPoint and many other formats. You send files as `multipart/form-data` and get a PDF back, and the README says you don't manage Chromium, LibreOffice or fonts yourself. From its docs, for the 9.x series:

```
docker run --rm -p 3000:3000 gotenberg/gotenberg:9
curl --request POST http://localhost:3000/forms/chromium/convert/html \
  --form files=@index.html --form files=@style.css -o out.pdf
```

We haven't run Gotenberg for this article; the commands are from its docs. From Node, any HTTP client can send the same form.

**Where it fits:** you want PDFs as a service for several apps or languages, or you don’t want a browser in your app’s image. Its Chromium route has the limits of any Chromium print. Its docs say Chromium prints the page as soon as it loads, so a page that draws later with JavaScript needs a wait.

**Where it doesn’t:** you now run, update and secure a service, and its docs have a “Secure your deployment” guide for that. The project is on version 9 and keeps the 8.x docs and image available, so check which version an example was written for.

## 7. HTML template to PDF in Node.js: render, then print

To convert HTML to PDF in Node.js from a template, you don’t need a separate route. A template engine (Handlebars, EJS, a template literal or a React server render) produces an HTML string, and the PDF step takes that string like any other page:

```
import { chromium } from "playwright";

const invoice = { number: 1042, lines: [{ name: "Widget", price: 120 }, { name:
"Gadget", price: 80 }] };
const html = `<!doctype html><meta charset="utf-8">
<style>@page { size: A4; margin: 20mm }</style>
<h1>Invoice ${invoice.number}</h1>
<ul>${invoice.lines.map((l) => `<li>${l.name}: ${l.price}</li>`).join("")}</ul>`;

const browser = await chromium.launch();
const page = await browser.newPage();
await page.setContent(html);
await page.pdf({ path: "invoice.pdf", preferCSSPageSize: true });
await browser.close();
```

We ran this with Playwright 1.65 and the installed Chrome (`channel: "chrome"`) and got one A4 page. `preferCSSPageSize` makes Playwright use the `@page` size rather than `format`, as its docs describe. Escape the values the way your template engine does for any web page. Playwright’s docs say `setContent` calls `document.write()` internally and inherits its behaviour, so if the template pulls in many files, write the HTML to disk and `goto()` it, or serve it. For React in particular, react to pdf compares the approaches.

## 8. Where papyrus fits

papyrus is a toolkit by n-uf for documents rendered as HTML and exported as PDF. It's the Playwright route, specialised for documents laid out as fixed sheets. You write the document as HTML, each element with the class `pdf-page` is one sheet on screen and one page in the PDF, and the `papyrus pdf` command prints each document listed in a config file with headless Chromium through Playwright. The exporter compares the number of page elements, the PDF's page count and the `pageCount` you expect, and writes no file when they disagree.

For plain HTML, point `baseUrl` at a folder with a `file:` URL and no server is needed. Otherwise the server that renders the documents must already be running. `papyrus pdf --check` reports committed PDFs whose sources changed, `PAPYRUS_CHROMIUM_PATH` prints with an installed Chrome, and `renderPdf()` from `@n-uf/papyrus/pdf` does the same job from Node code. Playwright is an optional peer dependency. papyrus doesn't draw PDFs, and it isn't a converter for any page you throw at it: it suits reports, résumés and invoices that you lay out as sheets. The html to pdf a4 page has the details.

papyrus is young. Its versions are dates ( `YY.M.R` ), not semantic versions, so any release may break things. Pin an exact version and read the CHANGELOG before you upgrade.

---

## 9. What to remember

- To convert HTML to PDF in Node.js, use headless Chromium: Playwright or Puppeteer, or Gotenberg if you want it as a service.
- PDFKit and pdfmake draw documents from code. Their READMEs list no HTML input.
- html-pdf wraps PhantomJS, and it's archived and deprecated.
- A template is just a way to make an HTML string. Print the string.
- Whichever route you pick, count the pages in the PDF you get.

## Questions people ask

### What is the best HTML to PDF converter for Node.js?

For HTML and CSS that should print the way a browser shows them, Playwright or Puppeteer with headless Chromium. Both print with the page's print CSS, and npm lists their latest releases on 5 and 10 November 2026. Choose PDFKit or pdfmake only if you'd rather build the document in code.

### Can PDFKit or pdfmake convert HTML to PDF?

Not according to their READMEs. PDFKit's features are vector graphics, text, fonts, images, tables and the like, drawn with calls, and pdfmake builds the PDF from a document definition object. If your input is HTML, use a browser route.

### Is html-pdf still maintained?

No. Its GitHub page says the repository was archived by the owner on Jul 8, 2026 and is read-only, and its README says it isn't maintained because PhantomJS was deprecated. On npm, its latest version, 3.0.1 from 7 May 2021, is marked deprecated.

### How do I generate a PDF from an HTML template in Node.js?

Render the template to an HTML string with your template engine, call `page.setContent(html)`, then `page.pdf()`. The browser only sees HTML, so the engine doesn't matter. Playwright's docs say `setContent` calls `document.write()`, so for a page with many assets, write it to a file or serve it and use `goto()`.

### Do I need Docker to convert HTML to PDF in Node.js?

No. Playwright and Puppeteer run a browser from your Node process. Gotenberg is the one that needs Docker, because it ships as a Docker image with an HTTP API; in return, there's no browser to manage inside your app.

## Sources


1. Playwright README ↗ checked 11 Nov 2026
2. Playwright: Page, page.pdf() ↗ checked 11 Nov 2026
3. Playwright: PDF Export (MCP tool) ↗ checked 11 Nov 2026
4. Playwright: Library ↗ checked 11 Nov 2026
5. Playwright for Python: Installation ↗ checked 11 Nov 2026
6. npm registry: playwright ↗ checked 11 Nov 2026
7. Puppeteer: home page ↗ checked 11 Nov 2026
8. Puppeteer: PDF generation guide ↗ checked 11 Nov 2026
9. npm registry: puppeteer ↗ checked 11 Nov 2026
10. Gotenberg README ↗ checked 11 Nov 2026
11. Gotenberg docs: introduction (9.x) ↗ checked 11 Nov 2026
12. Gotenberg: Convert HTML to PDF (9.x) ↗ checked 11 Nov 2026
13. GitHub: gotenberg/gotenberg releases ↗ checked 11 Nov 2026
14. PDFKit README ↗ checked 11 Nov 2026
15. npm registry: pdfkit ↗ checked 11 Nov 2026
16. pdfmake README ↗ checked 11 Nov 2026
17. npm registry: pdfmake ↗ checked 11 Nov 2026
18. GitHub: marcbachmann/node-html-pdf (html-pdf) ↗ checked 11 Nov 2026
19. npm registry: html-pdf ↗ checked 11 Nov 2026
20. papyrus README (@n-uf/papyrus v26.11.1) ↗ checked 11 Nov 2026
21. papyrus CLI help (@n-uf/papyrus v26.11.1, src/pdf/cli.js) ↗ checked 11 Nov 2026